ISA 401: Business Intelligence & Data Visualization

08: Structured Data from APIs

Fadel M. Megahed, PhD

Raymond E. Glos Professor in Business
Farmer School of Business
Miami University

@Fadel Megahed
fmegahed
fmegahed@miamioh.edu
Automated Scheduler for Office Hours

Fall 2026

Before We Start

1 Join today's Zoom

Use the Zoom link in the **Week 0 Module** on Canvas. **Mute** your mic and **turn off** your video.
Log in from the **lab's computer** so you can **share your screen** if you need help with an error.

2 Open the project, start the markdown

GitHub Desktop → `business_intelligence` → **Fetch origin**, then open `business_intelligence.Rproj`.
New R Markdown, title *Class 08*, delete everything below the setup chunk, save as **markdowns/class08.Rmd**.

3 Install one package

In the **Console** (not in the markdown): `install.packages("httr2")`
`jsonlite` and `dplyr` are already installed from Classes 04 and 07.

4 Request your USAJOBS key now

Fill the form at **developer.usajobs.gov/apirequest** with your Miami email, then **confirm the email** they send.
The key arrives in a later email. We use it in Part 3, about 30 minutes from now.

Done early? **Help the person next to you.** Stuck? **Share your screen on Zoom** and we fix it together.

Where We Left Off

Class 04: "a web-served JSON file"

```
crashes = jsonlite::fromJSON(
  "https://data.cincinnati-oh.gov/resource/rvmt-pkmq.json?$limit=100"
)
```

We pasted an address and got 100 rows. We did not discuss the `?$limit=100` at the end of it.

Today: the same address is an API

```
crashes = httr2::request("https://data.cincinnati-oh.gov/resource/rvmt-pkmq.json") |>
  httr2::req_url_query("$limit" = 100) |>
  httr2::req_perform() |>
  httr2::resp_body_string() |>
  jsonlite::fromJSON()
```

Everything after the `?` is a set of query parameters. Today you write them yourself.

Both return the same 100 rows. We use the `httr2` version today because it can also send a key, send headers, and report the status code. `jsonlite::fromJSON()` on an address cannot.

Learning Objectives for Today's Class

- **Define** what an API is and why APIs matter for data analysts.
- **Construct** API requests and handle authentication.
- **Parse** JSON responses into data frames.

Part 1 14 min

What is an API? The steps of a request, shown on Cincinnati's crash data (no key). The parts of an API URL. Status codes. The Class 06 chat app as an API client.

Part 2 15 min

One key in the URL. FRED: get a key, store it in `.Renviron`, read the documentation, request Ohio job postings.

Part 3 25 min

Key in headers. USAJOBS: reading documentation, nested JSON to a data frame, federal Data Analyst jobs. Then the same pipeline calls OpenAI.

The same httr2 pipeline is used in every part. Each API adds one line to it: query parameters, then a key, then headers, then a body. The Cincinnati request is worked out in full in Appendix A1 and A2.

Part 1: What Is an API?

What Is an API?

Client

Your R session. It wants data it does not have: crash reports, a job postings index, federal job ads.

→

API: application programming interface

The documented contract between the two. Which requests you may send, how to write them, and what comes back. Written for programs, not for people clicking.

→

Server

Someone else's computer holding the data. It answers requests that follow the contract and refuses the rest.

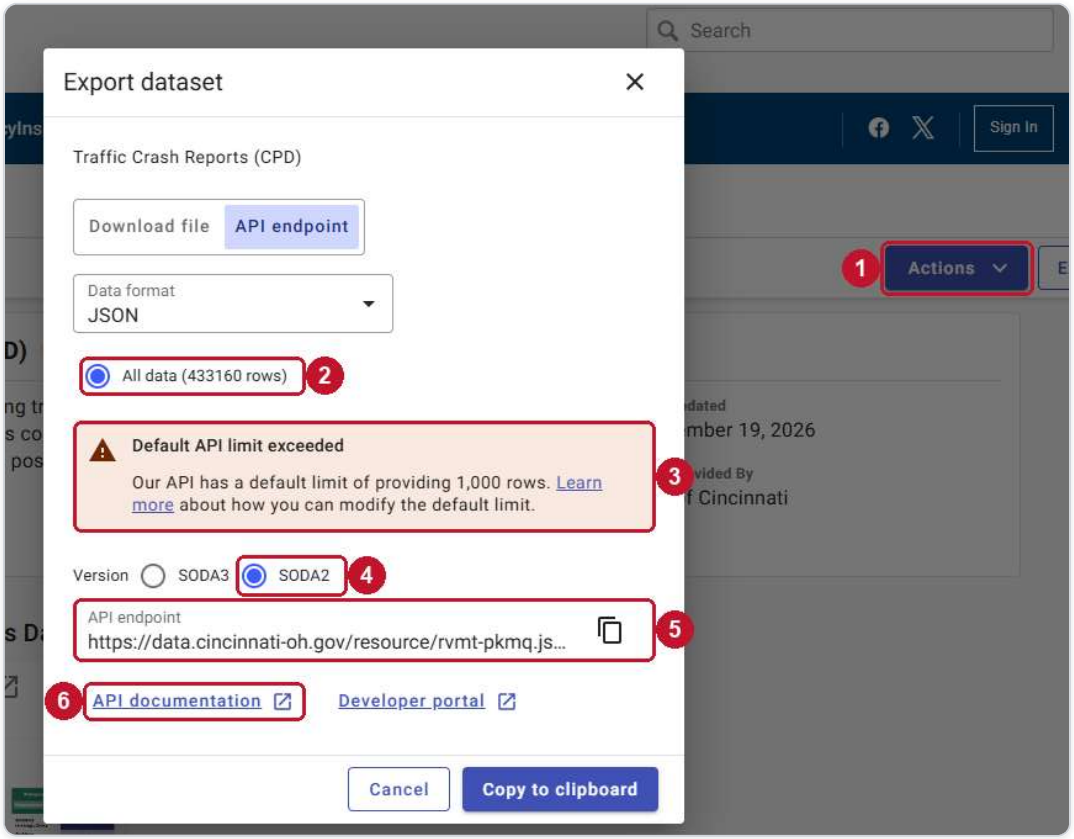
The interface is separate from the implementation. The owner can change the database behind the API. If the documented requests and replies stay the same, your code keeps working.

Controlled by the owner. The API exposes only what the owner chooses, to whom they choose, at the rate they choose. That is what keys and limits are for.

First example: the City of Cincinnati's crash data from Class 04. The next slides use it to show where an API address comes from and what happens when R sends a request.

Definition adapted from the MDN Web Docs glossary entry for API.

Documentation: Cincinnati Crash Data



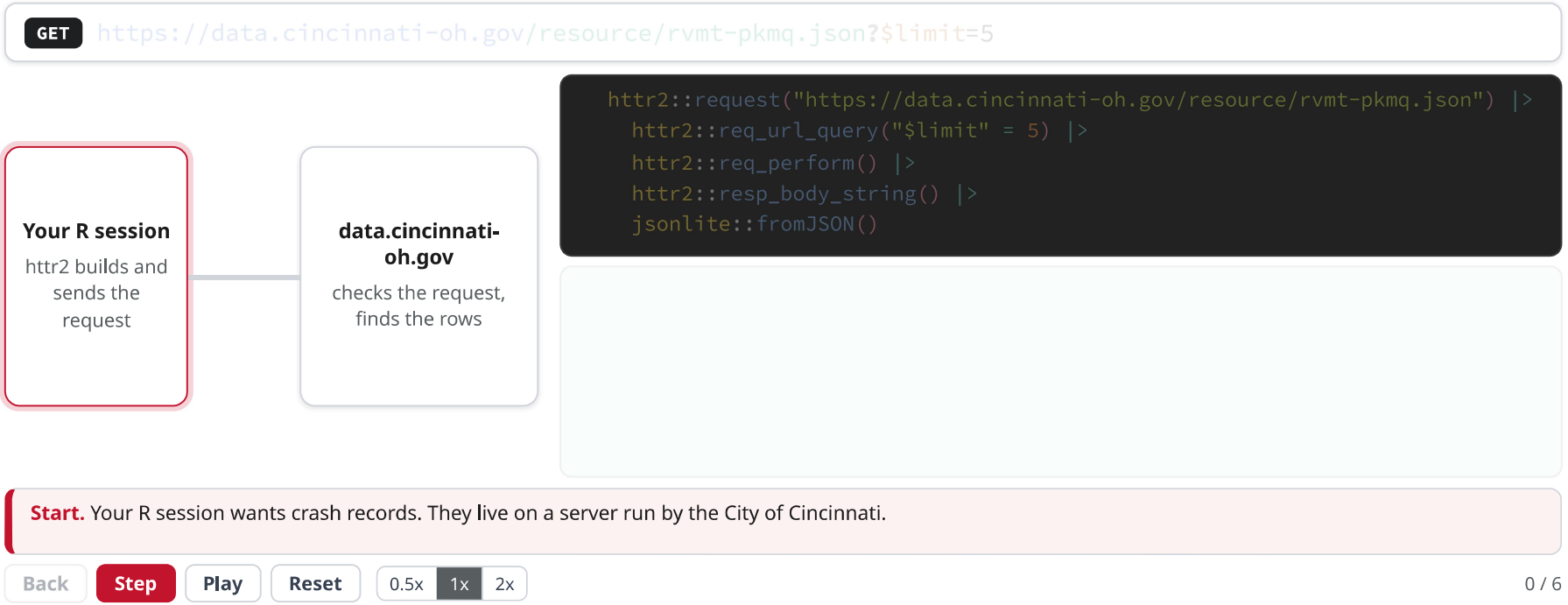
What this page tells us

1. On the dataset's page, **Actions** then **API** opens this dialog.
2. The table has **433,160 rows**. Too many to download for one question.
3. One request returns **1,000 rows** unless we ask for a different number.
4. Two API versions. **SODA3** requires authentication. **SODA2** does not, so we use it today.
5. The **API endpoint**: the address that every request to this table starts with.
6. The link to the **API documentation**, which lists the column names and the query options.

Open it yourself: Traffic Crash Reports (CPD) on the City of Cincinnati open data portal, then **Actions**, then **API**. Screenshot taken Sep 21, 2026.

The portal notes that this dataset is paused while the city changes record systems. The newest record is dated Aug 24, 2026.

A Request, Step by Step: Cincinnati



Anatomy of an API URL

```
https://data.cincinnati-oh.gov/resource/rvmt-pkmq.json?$limit=5&$order=crashdate DESC
```

Server
Whose computer. The City of Cincinnati's open data site.

Path (endpoint)
Which dataset on that server. `rvmt-pkmq` is the traffic crash table; `.json` is the format.

? and &
? starts the question. **&** separates one parameter from the next.

name=value
Query parameters. Each API defines its own parameter names and lists them in its documentation.

This server's parameters: `$limit` how many rows, `$order` sorted how, `$where` which rows, `$select` which columns. They correspond to the SQL clauses from Class 06 and the dplyr verbs from Class 07, and Socrata documents each one (LIMIT, ORDER BY, WHERE, SELECT). **The $ is part of the name on this server:** a request with `limit=5` is answered with 400, "Unrecognized arguments [limit]".

Status Codes

`200` OK
The request was understood and the data is in the body.

`400` Bad Request
The server could not read your request: a misspelled or missing parameter.

`401` / `403` Unauthorized / Forbidden
The server does not know who you are, or knows and says no. Usually the key.

`404` Not Found
Nothing lives at that path. Check the address against the documentation.

`429` Too Many Requests
You passed the owner's rate limit. Wait, then ask less often.

`500s` Server errors
The problem is on their side. Your request may be fine; try again later.

Read the first digit: 2 means success, 4 means a problem with your request, 5 means a problem on the server. `httr2::req_perform()` stops with an R error on any 4xx or 5xx reply, and the error message starts with the code. Meanings from the MDN HTTP status code reference.

Three Uses of the Word API

1 Data from a web server

A request over the internet, a reply in JSON.
Today's topic.

The Cincinnati request you just saw. Next: FRED economic series and USAJOBS postings.

2 A package's functions

The documented functions of a package are its API: names, arguments, and what each returns.

You used one last week: the dplyr function reference. No network involved.

3 A model behind a server

You send text, a model on someone else's computer sends text back. Same request and reply as use 1.

The Class 06 chat app works this way. See the next slide.

What the three share: documentation that says what you may ask for and what you get back, so you never need to see how the other side is built.

Your Chat App Is an API Client

You ask | The app sends a request | The model writes SQL | The app runs the SQL | The key

Job Scout Chat, in your browser

Which states have the most remote postings?

What you saw in Class 06

A question typed in plain English, and a table a few seconds later.

`app.R` is a short R script with no language model inside it. Something else interprets the question.

You ask. A question in English goes into the app you ran and deployed last week.

Back | Step | Play | Reset | 0.5x | 1x | 2x

0 / 4

12 / 39

Part 2: One Key in the URL

Get a FRED API Key

03:00

1 Create a free account

Go to **fredaccount.stlouisfed.org/apikeys**. It sends you to the sign-in page; create a free account with your Miami email.

2 Request the key

On the **API Keys** page click **Request API Key**. The form asks why you want it: *"Coursework for ISA 401 at Miami University"* is a complete answer.

3 Leave the page open

The key is 32 letters and digits, shown on the page right away. **Do not paste it into your markdown.** The next slide says where it goes.

Why a key at all? FRED's data is free, but the key tells the St. Louis Fed who is asking, so it can limit how fast any one person asks and switch off a key that misbehaves. Details: FRED API Keys documentation.

Keep the Key Out of Your Code

1. In the **Console**, open the project's `.Renviron` file:

```
usethis::edit_r_environment("project")
```

2. Add one line, no quotes, no spaces, then save. (This is a made-up key.)

```
FRED_API_KEY=abcdefghijklmnopqrstuvwxyz123456
```

3. Session → Restart R. R reads `.Renviron` only when it starts.

4. Check it, and make sure Git ignores the file:

```
Sys.getenv("FRED_API_KEY")   # shows the key, so: Console only
usethis::use_git_ignore(".Renviron")
```

Key typed in class08.Rmd

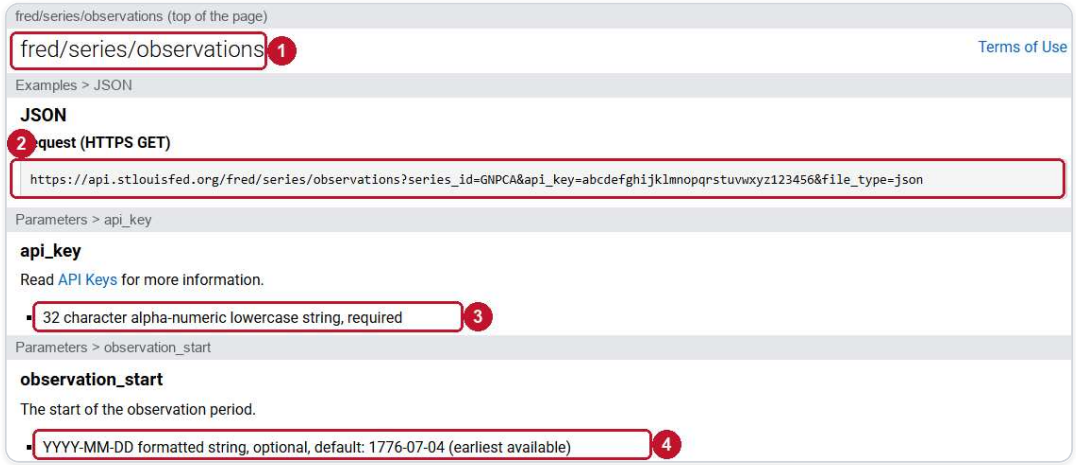
`class08.Rmd` → commit → push → **public on GitHub**, along with the key. This is what happened to the class OpenAI key on Sep 14.

Key in .Renviron

`class08.Rmd` says only `Sys.getenv("FRED_API_KEY")`. The value stays in `.Renviron`, which `.gitignore` keeps out of every commit. **The code is public, the key is not.**

Same recipe as the OpenAI key in Class 07. You will add a second line for USAJOBS in Part 3.

Documentation: FRED



What this page tells us

1. The **path** for the values of one series: `fred/series/observations`.
2. An **example request**. Server, path, then three parameters: `series_id`, `api_key`, `file_type`. The key shown is a made-up example.
3. `api_key` is **required**, and it is a query parameter. That is where our key goes.
4. `observation_start` is **optional**: a date as `YYYY-MM-DD`. Without it we get the whole history.

The full page: fred/series/observations in the FRED API documentation. Four parts of one long page are shown. Screenshot taken Sep 21, 2026.

A Request, Step by Step: FRED

GET https://api.stlouisfed.org/fred/series/observations?series_id=IHLIDXUSOH&file_type=json&api_key
•••• from .Renviron

Your R session
httr2 builds and sends the request

api.stlouisfed.org
checks the key, then finds the series

```
httr2::request("https://api.stlouisfed.org/fred/series/observations") |>
  httr2::req_url_query(
    series_id = "IHLIDXUSOH",
    api_key = Sys.getenv("FRED_API_KEY"),
    file_type = "json"
  ) |>
  # nothing has been sent yet
  httr2::req_perform() |>
  httr2::resp_body_string() |>
  jsonlite::fromJSON()
```

A different server. FRED, run by the Federal Reserve Bank of St. Louis, holds the Ohio job postings index from Class 04.

Back | Step | Play | Reset | 0.5x 1x 2x

0 / 7

Demo: Ohio Job Postings from FRED

Question. In Class 04 we downloaded this series as a CSV by hand. Can the API give us the same numbers, as a data frame, every time we knit?

The reply | The table | The plot | Your turn

```
 [1] "realtime_start"    "realtime_end"      "observation_sta
 [4] "observation_end"   "units"             "output_type"
 [7] "file_type"         "order_by"          "sort_order"
[10] "count"             "offset"            "limit"
[13] "observations"
```

```
[1] 1848
```

A list, not a table. `jsonlite::fromJSON()` returned 13 named elements. Most describe the request. One, `observations`, is the data frame we came for, with **1,848** rows. Which bracket from Class 04 pulls one element out of a list?

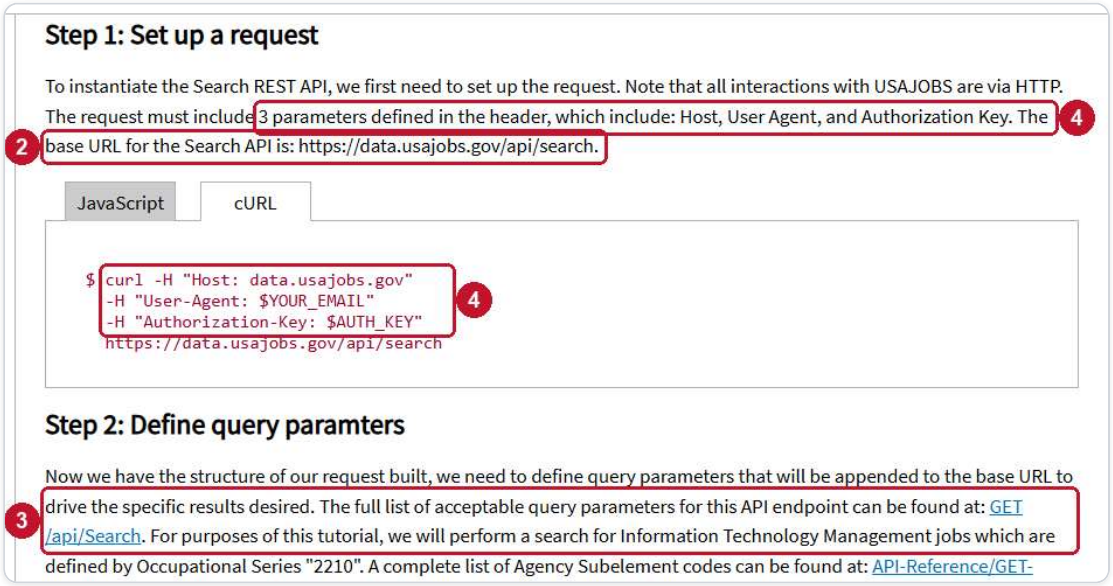
Part 3: Key in Headers

Reading API Documentation

Five questions for any API

1. Do I need a key, and how do I get one?
2. What is the base URL and the path?
3. Which query parameters exist?
4. Does the key go in the URL or in **headers**?
5. What are the limits?

Before the five: check whether an R package already exists for the API. Then open the **USAJOBS Search Jobs tutorial** and find the answers. The red numbers on the screenshot match the questions.



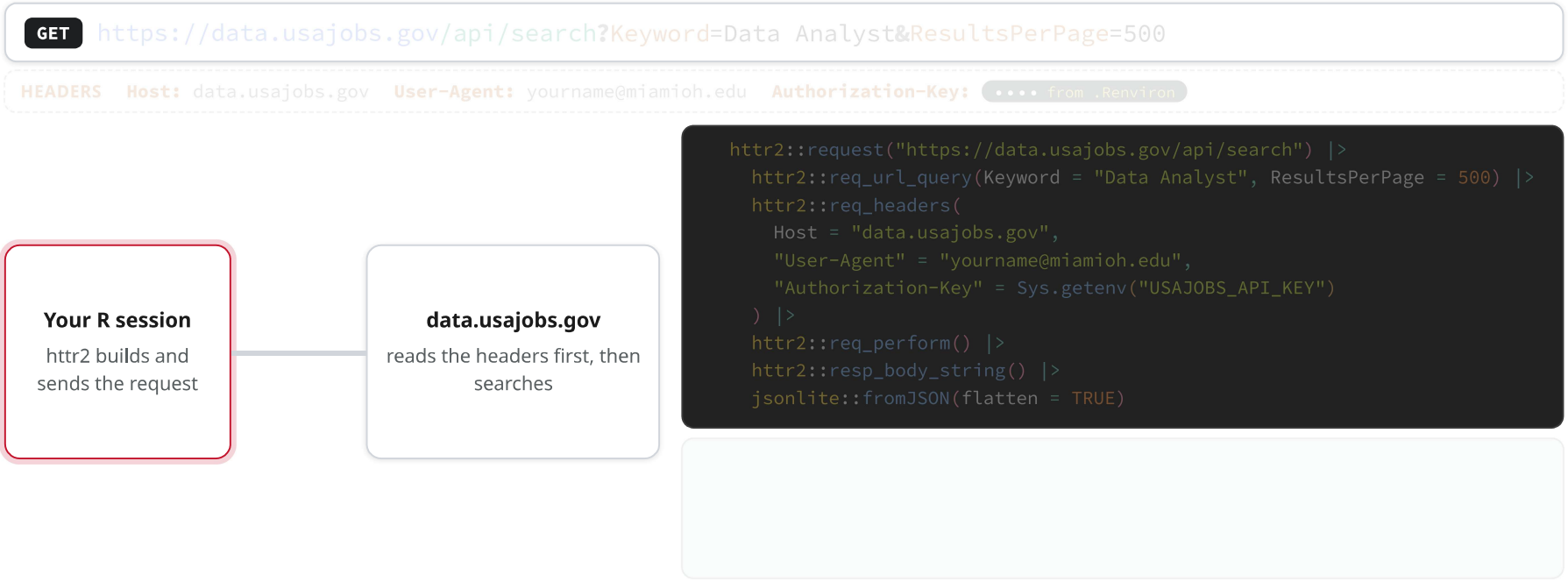
USAJOBS Developer Site, Tutorials, Search Jobs (cURL tab). Screenshot taken Sep 21, 2026.

What the USAJOBS Pages Say

1. **Key.** Free. Requested with a form and sent by email.
2. **Base URL and path.** `https://data.usajobs.gov/api/search`
3. **Query parameters.** `Keyword`, `LocationName`, `ResultsPerPage`, and many more, listed in the GET /api/Search reference.
4. **Where the key goes.** In **headers**, three of them: `Host` (data.usajobs.gov), `User-Agent` (your email), `Authorization-Key` (your key).
5. **Limits.** "The number of results returned per request defaults to 25 but can be defined in your request up to 500 per request."

What is different from FRED: the key is not a query parameter. It is sent in a header, together with two other headers that identify the sender.

A Request, Step by Step: USAJOBS



Demo: Data Analyst Jobs on USAJOBS

Question. How many federal postings match "Data Analyst" right now, and what does the reply look like?

```
[1] "SearchResultCount"    "SearchResultCountAll"
[3] "SearchResultItems"    "UserArea"
```

```
[1] 224  74
```

```
[1] "MatchedObjectId"
[2] "RelevanceRank"
[3] "MatchedObjectDescriptor.PositionID"
[4] "MatchedObjectDescriptor.PositionURI"
[5] "MatchedObjectDescriptor.OrganizationName"
```

Today: **224** postings, **74** columns. The count changes daily; yours may differ from your neighbor's by tomorrow.

Two levels down. The reply is a list, `SearchResult` is a list inside it, and `SearchResultItems` inside that is the table.

flatten = TRUE. Each posting is itself nested JSON. `jsonlite::fromJSON(flatten = TRUE)` spreads the nested fields into ordinary columns and joins the names with dots. That is where the long names come from.

From Nested JSON to a Data Frame

Question. The table has 74 columns with names like `MatchedObjectDescriptor.PositionTitle`. How do we get to five readable ones?

usajobs_df: 74 columns, four shown

~~MatchedObjectId~~	MatchedObjectDescriptor. PositionTitle	MatchedObjectDescriptor. DepartmentName	MatchedObjectDescriptor. PositionLocationDisplay
~~883905900~~	DATA ANALYST	General Services Adm	Multiple Locations
~~883911800~~	Data Analyst	Department of State	Washington, District
~~883741500~~	Program Analyst (Dat	Department of Homela	Washington, District
~~885259500~~	Program Analyst (Dat	Department of Homela	Washington, District
~~885121100~~	Management and Progr	Department of the In	Boise, Idaho

↓ `dplyr::select(new_name = old_name, ...)`

jobs_small: the same five postings

title	department	location
DATA ANALYST	General Services Administration	Multiple Locations
Data Analyst	Department of State	Washington, District of Columbia
Program Analyst (Data Analyst)	Department of Homeland Security	Washington, District of Columbia
Program Analyst (Data Analytics)	Department of Homeland Security	Washington, District of Columbia
Management and Program Analyst (Da	Department of the Interior	Boise, Idaho

Copy

```
jobs_small = usajobs_df |>
  dplyr::select(
    title = MatchedObjectDescriptor.PositionTitle,
    department = MatchedObjectDescriptor.DepartmentName,
    agency = MatchedObjectDescriptor.OrganizationName,
    location = MatchedObjectDescriptor.PositionLocationDisplay,
    closes = MatchedObjectDescriptor.ApplicationCloseDate
  )
```

Practice: Federal Data Analyst Jobs

05:00

Task Answer Discussion

Two questions on `jobs_small`

1. How many federal postings match "Data Analyst" today?

2. Which three departments have the most of them?

Work in pairs, in `class08.Rmd`. Your request must run from a fresh knit, with the key coming from `.Renviron`.

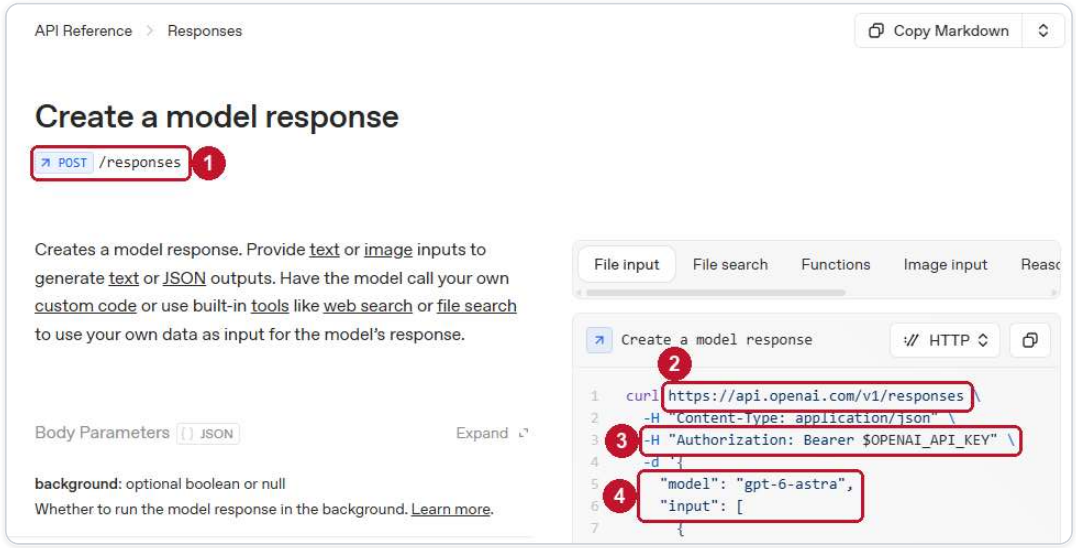
Key not here yet?

Pair with someone whose key has arrived, or answer these with your FRED key instead:

1. How many daily observations does `IHLIDXUSOH` have since 2026-01-01?

2. On which three dates in 2026 was the index highest?

Documentation: OpenAI

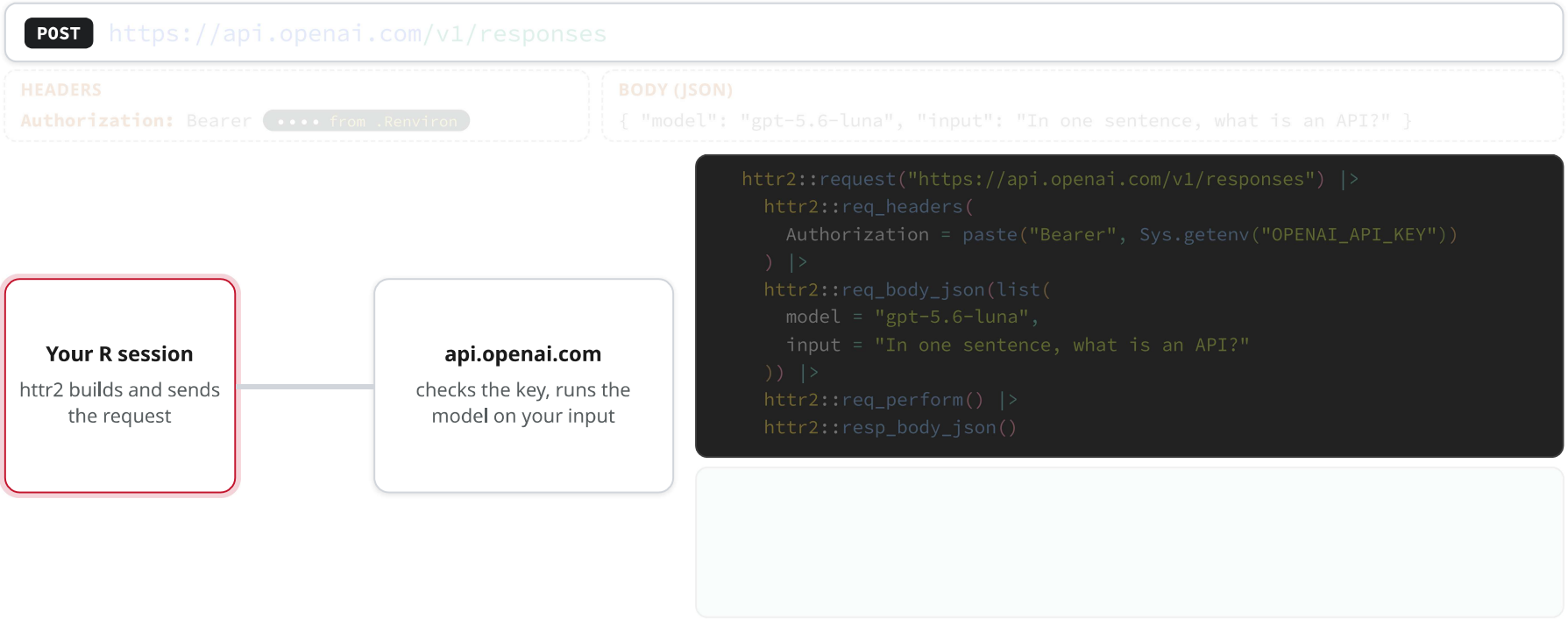


What this page tells us

1. The method is **POST** and the path is `/responses`. The first three APIs today used GET.
2. The **address**: `https://api.openai.com/v1/responses`. No query parameters.
3. The key goes in a **header**: `Authorization: Bearer` followed by the key.
4. The **body** is JSON with two fields we need: `model` and `input`. The page's example sends a file; `input` can also be plain text.

The full page: Create a model response in the OpenAI API reference. Screenshot taken Sep 21, 2026.

A Request, Step by Step: OpenAI



Calling OpenAI with httr2

Question. Part 1 showed that the chat app sends requests to OpenAI. Can we send one ourselves with the same httr2 pipeline?

```
reply = httr2::request("https://api.openai.com/v1/responses") |>
  httr2::req_headers(
    Authorization = paste("Bearer", Sys.getenv("OPENAI_API_KEY"))
  ) |>
  httr2::req_body_json(list(        # new: a body
    model = "gpt-5.6-luna",
    input = "In one sentence, what is an API?"
  )) |>
  httr2::req_perform() |>
  httr2::resp_body_json()

reply$output[[1]]$content[[1]]$text
```

```
An API (Application Programming Interface) is a set of rules
and tools that allows different software applications to
communicate with each other.
```

What is new. `httr2::req_body_json()` sends data **with** the request. A request with a body is a **POST**; everything before this slide was a **GET**. GET asks, POST sends.

The header. `Authorization: Bearer` followed by the key, as shown in Part 1. USAJOBS used a different header name; each API's documentation specifies its own.

How this relates to the app. Our `app.R` calls `ellmer::chat_openai(model = "gpt-5.6-luna")` and hands it to querychat. Packages like ellmer build and send this request for you.

The sentence on the left is the model's reply when these slides were built; yours will be worded differently. Reference: OpenAI API, create a response.

Our Class Git + GitHub Ritual

1 Knit

Click **Knit**. `class08.html` re-runs every chunk top to bottom and saves the page next to your markdown.

If the knit fails, a chunk depends on something you only ran in the Console. Fix the markdown, not the Console.

→

2 Commit

GitHub Desktop lists today's work: `markdowns/class08.Rmd` and `class08.html`. Nothing else.

Today you added two keys to `.Renviron`. If `.Renviron` is in the list, stop: uncheck it and run `usethis::use_git_ignore(".Renviron"`
Summary: `Add class 08 markdown on APIs`.

→

3 Push and verify

Push origin, then open the repo on GitHub and read `class08.Rmd` there.

Search the page for your key. You should find `Sys.getenv()` and nothing else.

If it is not pushed, it is not done. Your code contains the name of the key. The value stays on your PC.

Recap

Summary of Main Points

By now, you should be able to do the following:

- **Define** an API as a documented contract: you send a request the documentation describes, the server returns a status code and, when it says 200, data.
- **Construct** a request with `httr2::request()`, `httr2::req_url_query()`, and `httr2::req_headers()`, with every key read from `.Renviron` by `Sys.getenv()`.
- **Parse** the JSON reply with `jsonlite::fromJSON()`, find the table inside the list, and hand it to the dplyr verbs.

Next class (Mon, Sep 28): Web Scraping I, for the pages that have no API.

Before You Go

Join at **https://www.menti.com/al37nbpvnzpg**

Before you go: one thing from today you would like clarified.

Review and Clarification

- **Class Notes**: Take some time to revisit your class notes for key insights/concepts.
- **Zoom Recording**: The recording of today's class will be made available on Canvas approximately 3-4 hours after the session ends.
- **AI Recaps:** make your own with Gemini's NotebookLM, as I did for Class 01 (see the right).
- **Questions**: Do not let confusion pile up.

Make your own recap in NotebookLM

- Go to notebooklm.google.com and sign in with your Miami Google account.
- Add today's slide deck as a source: paste the deck's address from Canvas as a **Website** source.
- Click **Audio Overview** for a podcast, or **Video Overview** for a narrated video. Both take a few minutes to generate.

What it looks like: the Class 01 video and podcast. Treat them as a study aid, not a substitute for the slides: the AI summarizes what is on the page and can misstate details.

🎯 Assignments 🎯

No new graded assignment today

Open assignments and their due dates are on the **Canvas Assignments page**. The practice questions on the right use the three APIs from today and the keys you now have.

Keep your keys private. The FRED and USAJOBS keys are registered to your email. They live in `.Renviron`, never in a markdown, a commit, a screenshot, or a chat message.

Before you leave: knit, then commit and push `markdowns/class08.Rmd` and `class08.html`. If it is not pushed, it is not done.

Ungraded Practice Questions

1. Cincinnati (Appendix A1 and A2). Which five `sna_neighborhood` values have the most records among the 1,000 most recent?

2. FRED. Get `IHLIDXUSOH` and `IHLIDXUS` since 2026-01-01. On how many days was Ohio's index above the national one?

3. USAJOBS. Change the request so it returns "Data Analyst" postings in Ohio (the documentation's `LocationName` parameter). How many are there?

4. USAJOBS. What are the typical minimum and maximum pay for today's postings? The pay columns are nested one level deeper; see Appendix A3.

5. OpenAI. Send your own one-sentence question. Then change `input` to ask for the answer in Spanish.

▶ **Answers**

Appendices

Not covered live. A1 for the bracket-to-verb translation, A2 when a deploy fails, A3 for the reading.

A1: Cincinnati Example in httr2

Question. How many traffic crashes has Cincinnati recorded since August 1, 2026?

Two ways to answer. Download every record and count rows in R, or ask the server to count and download one number. The table has 433,160 rows today.

What is one row? One person involved in a crash. Rows from the same crash share an `instanceid`, so counting crashes means counting **distinct** `instanceid` values, with a condition on `crashdate`.

Copy

```
cincy_url = "https://data.cincinnati-oh.gov/resource/rvmt-pkmq.json"

httr2::request(cincy_url) |>
  httr2::req_url_query(
    "$select" = "count(*) as records, count(distinct instanceid) as crashes",
    "$where" = "crashdate >= '2026-08-01'"
  ) |>
  httr2::req_perform() |>
  httr2::resp_body_string() |>
  jsonlite::fromJSON()
```

```
  records crashes
1    1801     865
```

Is the number reasonable? The newest record is dated Aug 24, so the request covers 24 days: about 36 crashes a day, from 1,801 records. The same request for all of 2025 returns 14,941 crashes, about 41 a day. Both values are text, like every column from this server.

A2: Practice: Build the URL

05:00

Task Answer Discussion

☑ **$limit** 5 ☐ **$where** crashdate >= 08/01/2026 ☐ **$order** crashdate DESC ☐ **$select** count(*)

https://data.cincinnati-oh.gov/resource/rvmt-pkmq.json?$limit=5 **Open in a new tab**

Copy

```
httr2::request("https://data.cincinnati-oh.gov/resource/rvmt-pkmq.json") |>
  httr2::req_url_query(
    "$limit" = 5
  ) |>
  httr2::req_perform() |>
  httr2::resp_body_string() |>
  jsonlite::fromJSON()
```

1. Pick any date in August 2026. How many crash records since that date? Let the **server** count.

2. Download the 1,000 most recent records. Which `crashseverity` is most common? Let **R** count.

Why the quotes? On this server the `$` is part of each parameter's name, and an R argument name cannot start with `$` unless it is quoted. FRED and USAJOBS use plain names, so no quotes are needed there. Tick boxes to build the request, open it in a browser tab to read the raw reply, then run the R version in `class08.Rmd`. The newest record in this table is dated 2026-08-24.

37 / 39

A3: Practice with USAJOBS Pay and Remotive

USAJOBS: typical pay

The column `MatchedObjectDescriptor.PositionRemuneration` is a **list column**: each cell holds a small data frame with `MinimumRange`, `MaximumRange`, and `RateIntervalCode` ("PA" means per year).

```
usajobs_df |>
  dplyr::select(
    title = MatchedObjectDescriptor.PositionTitle,
    pay = MatchedObjectDescriptor.PositionRemuneration
  ) |>
  tidyr::unnest(pay) |>
  dplyr::filter(RateIntervalCode == "PA") |>
  dplyr::summarize(
    min = median(as.numeric(MinimumRange)),
    max = median(as.numeric(MaximumRange))
  )
```

When these slides were built: median minimum $87,362, median maximum $117,525, across 223 per-year pay ranges.

Remotive: remote jobs, no key

1. Open remotive.com/api/remote-jobs?category=data&limit=25 in a browser tab. Read the **legal notice** at the top of the reply: it states what you may do with the data and asks you to credit Remotive and link back.
2. Write the same request with `httr2`. The table is the `jobs` element (13 columns when these slides were built).
3. The documented parameters are `category`, `company_name`, `search`, and `limit` (Remotive API documentation). Use `search` to find postings that mention SQL.
4. Which `candidate_required_location` values are most common?

Job data on this slide's exercise comes from Remotive. Its documentation advises at most four requests a day and blocks more than two a minute, so ask once, save the data frame, and work from the saved copy.

A4: Reading

Suggested

Get started with httr2

The package's own introduction: create a request, modify it, perform it, read the response. Today's pipeline, with more options.

Skim if curious

httr2: Wrapping APIs

How R packages such as ellmer are built on top of requests like today's, including a section on keeping secrets out of code.

Skim if curious

FRED API documentation

A well organized example of API documentation: every path, every parameter, an example request and reply for each.

Skim if curious

Socrata API endpoints and versions

Explains the two versions seen on the Cincinnati portal: the older one we used (no key) and version 3, which requires authentication.